

ImageLab Technical Reflection

CMPS4191: Advanced Web Technologies

University of Belize

Andres Hung & Jennessa Sierra

Sep 30, 2026

<https://github.com/andreshungbz/imagelab>

IMAGELAB	Title	ImageLab Technical Reflection
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Sep 30, 2026

Project Reflection

Version 1 of ImageLab comprised a full-stack single-page web application for generating image variants. The backend was implemented using Go and PostgreSQL. The Go API server provided the necessary HTTP endpoints, a single background worker, and file storage for uploaded images. It also provided the appropriate HTTP 202 acknowledgment. The PostgreSQL database provided tables for managing the job state and the metadata for images and image variants. These were controlled via SQL migrations. The frontend was implemented using vanilla HTML, CSS, and JavaScript, and it handled the UI and job state monitoring. Measurement testing was implemented using Python.

The frontend uses the event-emitter pattern to effectively manage UI state without frameworks. Here, a simple client-side disabled button is implemented to prevent duplicate POST requests. Future versions should implement proper idempotency.

The backend implements the asynchronous API pattern of short polling, where the frontend, after receiving an HTTP 202 acknowledgment, sends HTTP requests to the job status endpoint at a 1-second interval. With the constraint of a single worker, the limitations of this approach are evident not only in the additional resource usage of sending multiple requests, but also in the Queue Wait, Job Duration, and Polling Count measurements. With an artificial 3-second image-processing delay, jobs sent in quick succession result in longer durations. For a single job, the times are fine, but with a five-image burst, the Queue Wait, Job Duration, and Polling Count increase significantly. The rest of the measurements remained consistent with one another, as expected, since they were not dependent on position in the worker queue. Future versions can implement another asynchronous API pattern and/or implement multiple workers to address these issues in Version 1.

```

λ just measure-baseline
uv run --directory ./python measurement --count 1 --api-url="http://localhost:$PORT" --db-dsn="$IMAGELAB_DB_DSN" --image-path="src/measurement/pittsburgh.jpg"
ImageLab Measurements (Images Count: 1)
=====
Job # | Public ID | Ack Latency | Queue Wait | Processing Duration | Job Duration | Polling Count | Detection Delay
=====
1 | bb676206-7a50-40a4-9a83-16a092761874 | 24.7ms | 0.02s | 3.46s | 3.47s | 4 | 540.8ms
=====

```

```

λ just measure-concurrent
uv run --directory ./python measurement --count 5 --api-url="http://localhost:$PORT" --db-dsn="$IMAGELAB_DB_DSN" --image-path="src/measurement/pittsburgh.jpg"
ImageLab Measurements (Images Count: 5)
=====
Job # | Public ID | Ack Latency | Queue Wait | Processing Duration | Job Duration | Polling Count | Detection Delay
=====
1 | 51c7a25a-8419-4f4c-8461-d3edfda3b555 | 32.3ms | 14.09s | 3.46s | 17.54s | 18 | 524.8ms
2 | ecea731b-ae03-43da-862b-8ba74eebade | 31.3ms | 10.62s | 3.47s | 14.09s | 15 | 976.2ms
3 | 302a3e3f-93bf-4e79-b1a4-551dd538f147 | 27.8ms | 3.67s | 3.48s | 7.15s | 8 | 897.2ms
4 | c47f2cb1-1c3e-4469-9600-b97d50c1b1dd | 43.4ms | 0.19s | 3.48s | 3.67s | 4 | 340.6ms
5 | bc8a63b6-a144-4172-a43d-785b73422f33 | 28.1ms | 7.15s | 3.47s | 10.62s | 11 | 433.0ms
=====

```

Note that the ordering of the results may not be strictly increasing due to concurrent measurement execution.

The next page answers the required interpretation questions.

IMAGELAB	Title	ImageLab Technical Reflection
	Course	CMPS4191 Advanced Web Technologies
	Semester	2026-1
	Due Date	Sep 30, 2026

Required Interpretation Questions

- Explain why 202 can reduce acknowledgment latency without reducing image-processing work.**
 Because a 202 only indicates job acceptance, it can reduce acknowledgment latency since it doesn't depend on image processing. With a separate worker handling image processing, the handler can return a response without reducing the image-processing work.
- Explain why later jobs normally wait longer when five jobs share one worker.**
 One worker can work on only one job at a time, so when 5 jobs are queued, later jobs must wait longer.
- Explain how a one-second interval affects polling request count and detection delay.**
 The polling request count is directly related to the one-second interval, since a poll request is sent during that interval to check the job status. The one-second interval also affects the detection delay, as observing the job completion status depends on the last poll that receives the completed job status. That poll depends on the polling interval, processing time, and the worker claim interval.
- Identify at least one limitation that a later observation mechanism may address. Do not implement that mechanism in Assessment 1.**
 One limitation is the frequent requests sent by short polling, which causes unnecessary traffic and resource usage. Long polling can be a mechanism that addresses this.